SOFTWARE PARA SISTEMAS UBÍQUOS · UFG · MARCO 3

Presença

Segurança de ponta a ponta: canal cifrado, autenticação no broker e **todo dispositivo dizendo quem é — com prova**.

Alany Gabriely · 16/10/2026

O QUE FOI PEDIDO NA AULA

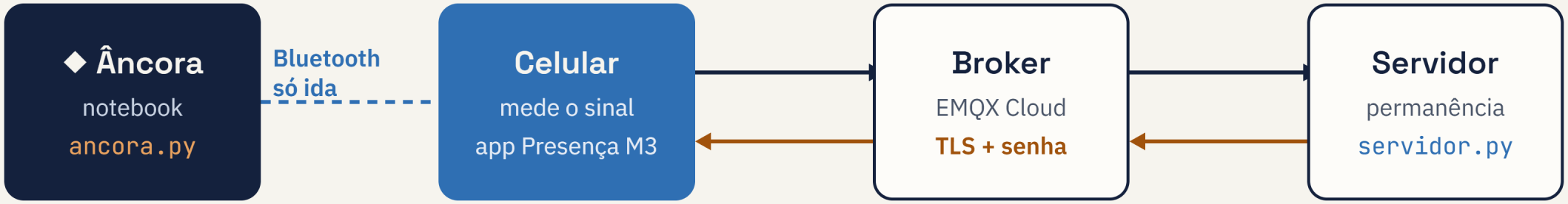
A entrega do Marco 3, item por item

Entrega do Marco 3	Slides	Situação
Segurança (o tema principal)	4–5	feito
Canal criptografado	8, 10	feito e testado
Autenticação MQTT	9–10	feito e testado
Todo dispositivo tem que falar quem é	11–17, 19	feito e testado
Fazer atuação e autenticação	18, 20	feito e testado
Processo inverso: produtor ↔ consumidor	18	feito
Fazer funcionar redes de sensores	21	1 âncora real

Além do pedido (slides 22–26): quantos aparelhos as âncoras atendem, Arduino/ESP32, distância pelo sinal.

O PROTÓTIPO VIROU O SISTEMA

Quatro peças, um rádio de verdade



1 · A âncora anuncia “PRS1 + token”. Não escuta ninguém.

2 · O celular mede a força do sinal e decide perto ou longe de 1 m.

3 · O broker leva as mensagens. Agora só com TLS e senha.

4 · O servidor calcula a aula e devolve estado e avisos.

ANTES DE PROTEGER: DO QUE PROTEGER

Quatro jeitos de enganar o sistema

1 · Ler no caminho

Alguém no mesmo Wi-Fi da faculdade lê as mensagens — e a senha, que vai na conexão.

→ **canal cifrado (TLS)**

2 · Entrar no broker

Qualquer pessoa conecta, lê todos os tópicos e publica o que quiser.

→ **autenticação MQTT**

3 · Falar em nome de outro

Quem tem a senha do app manda “estou na sala” pelo colega, ou finge ser o servidor.

→ **assinatura por dispositivo**

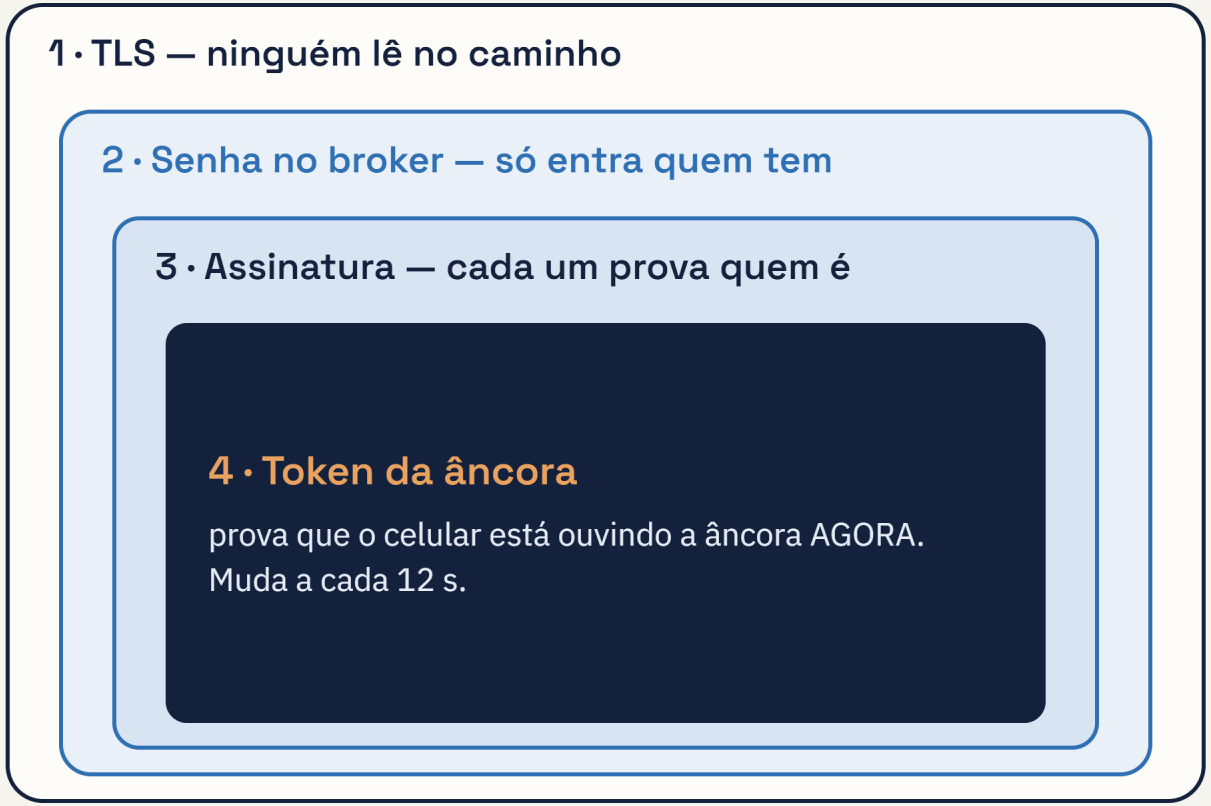
4 · Dizer que está na sala, de casa

O próprio aluno assina um heartbeat inventado ou repete um antigo.

→ **token da âncora**

A RESPOSTA INTEIRA EM UM DESENHO

Quatro camadas, de fora para dentro



Cada camada cobre o que a de fora não cobre

TLS esconde o conteúdo, mas não diz quem fala. A senha barra quem é de fora, mas é a mesma para todos os celulares.

O ponto chave

A senha do app vai **dentro do APK**. Quem descompilar tem. Por isso a identidade precisa estar **na mensagem**, não só na conexão.

Na mesma ordem dos ataques

1 ler · 2 entrar · 3 falar por outro · 4 fingir que está na sala.

O PROBLEMA, RODANDO · BROKER.HIVEMQ.COM

Até o Marco 2, o broker era uma praça pública

```
$ python ataques.py publico --ws
== 1. Broker PÚBLICO (broker.hivemq.com) ==
atacante tenta: conectar SEM usuário e SEM senha
CONSEGUIU — conectado
atacante tenta: assinar ufg/ssu/presenca/# e PUBLICAR um
heartbeat falso
CONSEGUIU — publicou em …/proto1/sala-pc/heartbeat e o broker
entregou de volta: escrever e ler, sem senha
```

Sem senha

Qualquer pessoa no mundo conecta.

Lê e escreve

O heartbeat falso foi aceito e entregue a quem assina o tópico.

E o wss da 8884?

Cifra o caminho, mas não diz quem fala. Criptografia sem autenticação não basta.

A DICA DO PROFESSOR: HIVEMQ

Por que o broker agora é o EMQX

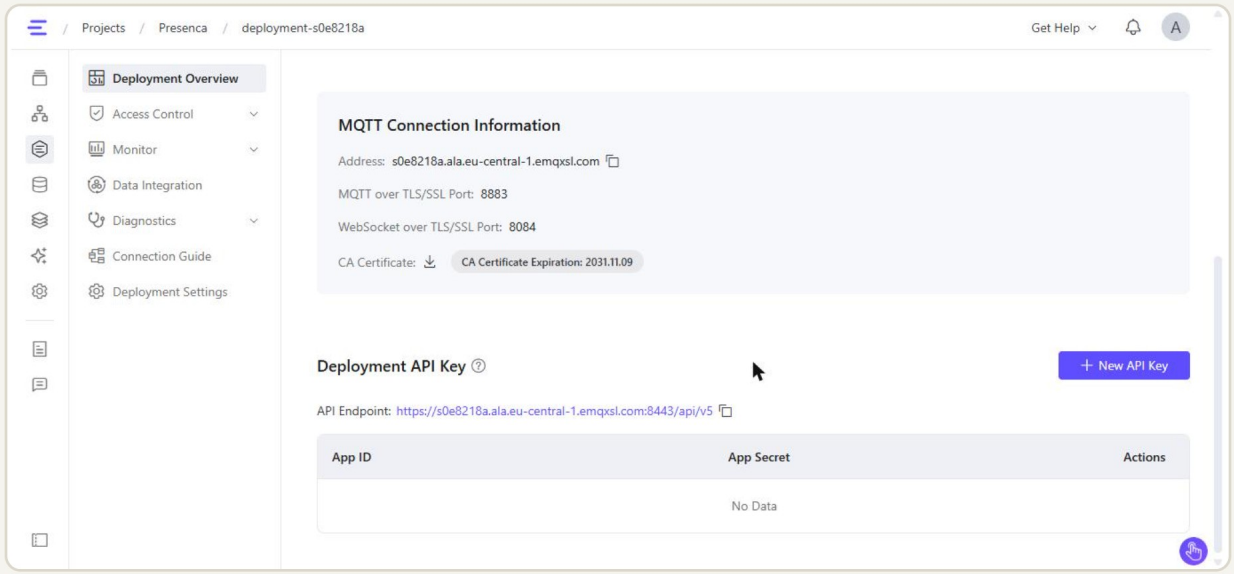
	HiveMQ público	HiveMQ Cloud	EMQX Cloud Serverless
Senha	nenhuma	sim	sim, por usuário
Só TLS	não (1883 aberta)	sim	sim (8883 e 8084)
Tópico por usuário (ACL)	não	só no pago	sim
Custo	grátis	cartão; ~US$ 249/mês após 15 dias	grátis, sem cartão
Limite de gasto	—	—	US$ 0: suspende, não cobra

Para o código, trocar de broker é trocar uma linha

O protocolo é o mesmo (MQTT 3.1.1). Endereço, portas e senhas ficam em **segredos.json**, fora do git; **configurar.py** leva o que o app precisa para dentro do APK.

CAMADA 1 · CANAL CRIPTOGRAFADO

TLS: o mesmo cadeado do HTTPS



Print do console: só existem as portas cifradas, 8883 e 8084.

1 · Cifra tudo

Quem está no Wi-Fi vê só bytes embaralhados — nem a senha aparece.

2 · Prova que o broker é o broker

O certificado é conferido. Ninguém se põe no meio fingindo ser ele.

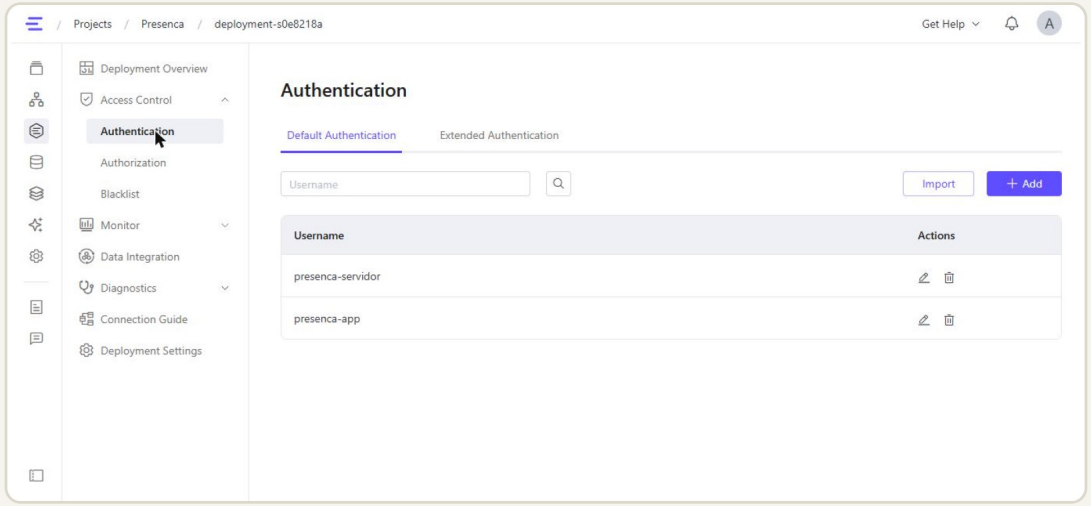
3 · Não há porta aberta

1883 nem existe. No código não há caminho sem TLS:

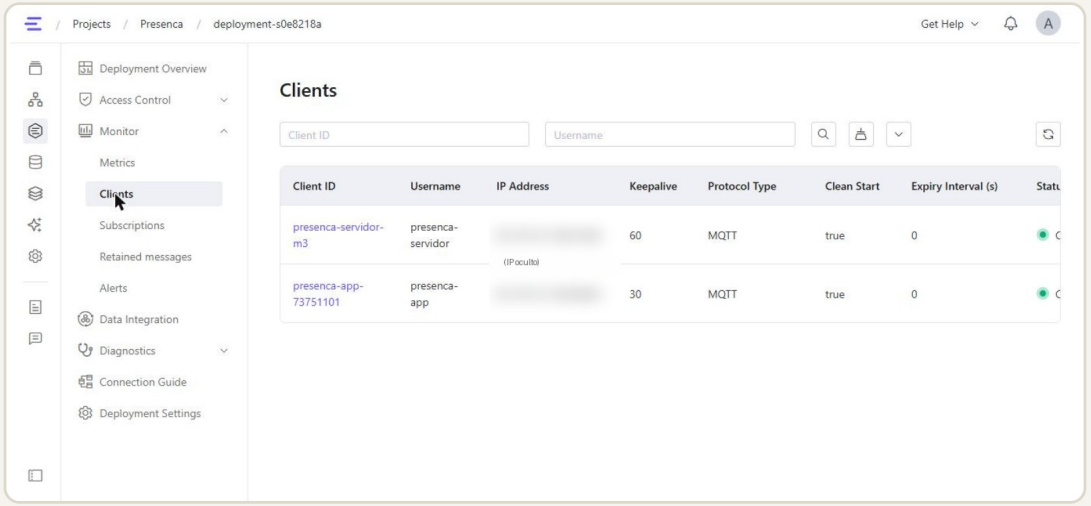
```
cliente.tls_set()  # servidor
useSSL: true      // app
```

CAMADA 2 · AUTENTICAÇÃO MQTT

Um usuário para cada papel



Cadastro: dois usuários. A senha nunca aparece no console.



No ensaio: cada conexão entra com o seu usuário (IP oculto).

presenca-servidor · só o notebook do servidor. Senha em segredos.json.

presenca-app · todos os celulares. A senha vai dentro do APK.

Senhas · 20 caracteres aleatórios, gerados pelo módulo secrets do Python.

PROVA DAS CAMADAS 1 E 2 · SAÍDA REAL

O intruso tenta entrar no EMQX

```
$ python ataques.py senha
tenta: MQTT aberto, sem TLS, porta 1883
BLOQUEADO — a conexão nem abriu
tenta: TLS na porta 8883, SEM usuário e senha
BLOQUEADO — o broker recusou: Bad user name or password
tenta: TLS com o usuário presenca-app e senha ERRADA
BLOQUEADO — o broker recusou: Bad user name or password
tenta: TLS com usuário e senha certos (para comparar)
CONECTOU — conectado
```

Sem TLS
a porta nem existe

Sem senha ou errada
o broker recusa na conexão

Com a senha certa
entra — o controle é a senha, não a sorte

Também testado pela porta **8084** (WebSocket): mesmo resultado.

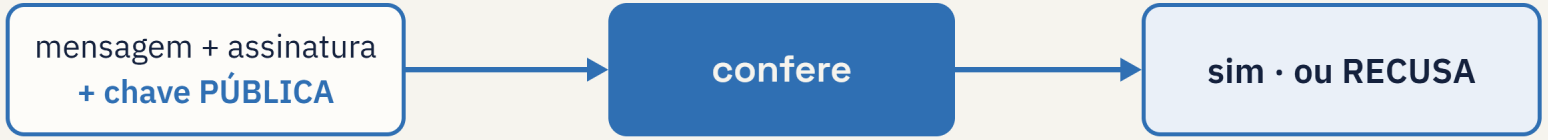
CAMADA 3 · TODO DISPOSITIVO TEM QUE FALAR QUEM É

Um carimbo que só o dono tem

ASSINAR · no aparelho



CONFERIR · em quem recebe



Por que Ed25519

Padrão (RFC 8032), chave de 32 bytes, assinatura de 64, rápido no celular. Biblioteca noble-ed25519 no app e cryptography no servidor.

Privada

nunca sai do aparelho. É o carimbo.

Pública

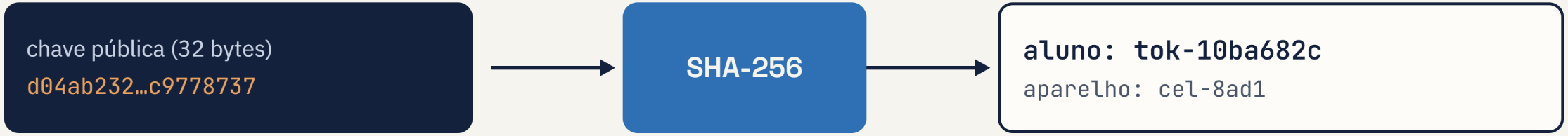
vai junto da mensagem. Qualquer um confere o carimbo.

Mudou 1 byte?

a assinatura não confere mais.

IDENTIDADE DO ESTUDANTE

O nome do aluno sai da própria chave



1 · Sem cadastro

O servidor recalcula o hash da chave que veio no envelope. Se der o mesmo nome, a chave é dele.

2 · Impossível de imitar

Para assinar como tok-10ba682c é preciso a chave privada que gerou esse nome.

3 · Privacidade

Nada de MAC, IMEI ou nome. É a decisão 5 do projeto, agora de verdade.

A chave privada fica no armazenamento seguro do Android (expo-secure-store, cifrado pelo Keystore). Valores do teste real que confere app e servidor.

SEGURANCA.ENVELOPE.V1

Toda mensagem viaja num envelope assinado

```
{
  "eventType": "seguranca.envelope.v1",
  "de": "tok-10ba682c",
  "chavePublica": "d04ab232…8737",
  "carga": "{\"eventType\":
    \"presenca.heartbeat.v2\", …}",
  "assinatura": "d102ad2b…a405"
}
assinatura = Ed25519(tópico + "\n" + carga)
```

de

quem diz ser. Só um rótulo: o que vale é a chave.

carga

a mensagem de antes, intacta, como texto. Os bytes exatos que foram assinados.

assinatura

cobre o **tópico** também: não dá para levar a mensagem do aluno A para o tópico do aluno B.

Nos dois sentidos

celular assina o heartbeat; servidor assina estado e aviso; admin assina a turma.

O QUE O SERVIDOR FAZ COM CADA HEARTBEAT

Quatro portas antes de contar um segundo

1

A assinatura confere?

com a chave do envelope e o tópico em que chegou

`SEM ASSINATURA`
`ASSINATURA FALSA`

2

Respeita o contrato?

JSON Schema presenca.heartbeat.v2

`FORA DO CONTRATO`

3

O nome bate com a chave?

hash(chave) tem de ser o aluno que diz ser

`IDENTIDADE TROCADA`

4

O token é de agora?

o que a âncora anuncia nesta janela de 12 s

`TOKEN VENCIDO`

Passou nas quatro → entra no log e conta para a permanência

Falhou em qualquer uma → x RECUSADA com o motivo na tela, e nada é somado. A ordem importa: primeiro quem é, depois onde estava.

TODO DISPOSITIVO TEM QUE FALAR QUEM É · RESPOSTA

Os quatro dispositivos, e a prova de cada um

Dispositivo	Como prova quem é	Quem confere	Se falhar
Celular do aluno	assina cada heartbeat; o nome é o hash da chave	servidor	IDENTIDADE TROCADA
Âncora	anuncia um token que só ela e o servidor sabem gerar	servidor	TOKEN VENCIDO
Servidor	assina estado e avisos; a chave dele vem fixada no app	celular	“mensagem falsa”
Celular do admin	assina a turma; o aluno guarda essa chave no pareamento por QR	celular do aluno	“turma recusada”

No broker
cada conexão diz o seu papel: usuário e senha.

Em cada mensagem
cada aparelho diz quem é: chave e assinatura.

CAMADA 4 · A ÂNCORA TAMBÉM DIZ QUEM É

O token só vale enquanto a âncora o anuncia

```
token = HMAC-SHA256( segredo da âncora, "anc-pc-1|" + ⌊hora ÷ 12 s⌋ )[0..3]
```

janela -5	-4 a -2	-1 · atraso	ATUAL · 12 s	+1 · relógio
recusada	só se veio da fila	aceita	aceita	aceita

1 · Ninguém combina nada

âncora e servidor usam o mesmo relógio e o mesmo segredo: calculam o mesmo token.

2 · O celular só repassa

não tem o segredo: não consegue inventar nem conferir um token.

3 · De casa não funciona

sem ouvir a âncora agora, não há o token de agora. Repetir um velho: vence.

JANELA 1 · SAÍDA REAL DA ÂNCORA

Um token novo a cada virada de janela

```
$ python ancora.py
== Presença · Marco 3 · âncora (este notebook) ==
Anunciando por Bluetooth como anc-pc-1.
Token novo a cada 12 s, assinado com o segredo (HMAC).

02:41:05  janela 149344105  token 45cb0d0e
          rádio: TRANSMITINDO
02:41:12  janela 149344106  token 42f6d47b
02:41:24  janela 149344107  token bf18fee7
```

janela
segundos desde 1970 ÷ 12. Sobe 1 a cada 12 s.

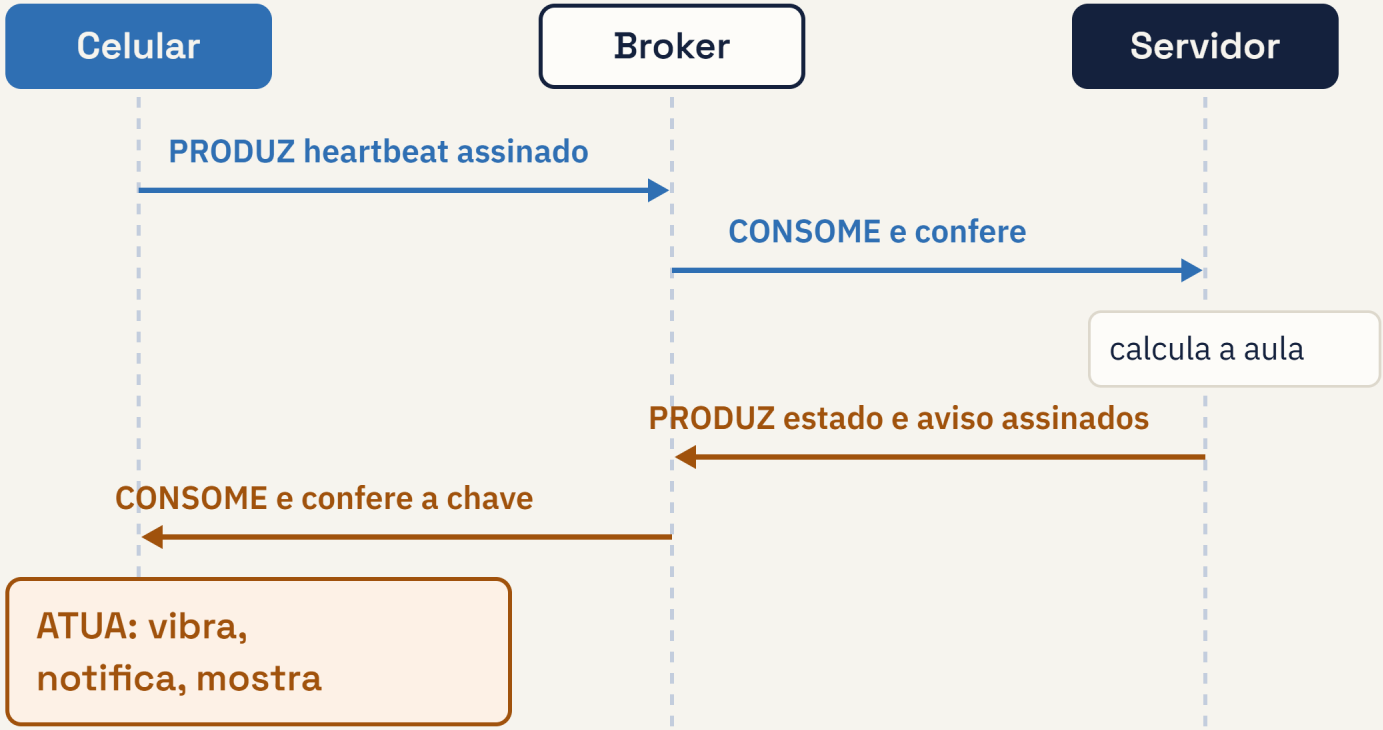
token
4 bytes do HMAC. Sem o segredo, impossível adivinhar o próximo.

02:41:12
a troca segue o relógio, não a hora em que a âncora ligou.

anúncio Bluetooth: "PRS1" | 01 | 45 cb 0d 0e   (empresa 0xFFFF, reservada para testes)

ATUAÇÃO · PROCESSO INVERSO: PRODUTOR ↔ CONSUMIDOR

Os dois lados produzem e consomem



Celular
produz o heartbeat e consome estado e aviso.

Servidor
consome o heartbeat e produz estado e aviso.

Atuação
no celular, só por mensagem comprovadamente do servidor. “Notificar antes de punir.”

Autenticação nos dois sentidos
quem produz assina; quem consome confere.

PROVA DA CAMADA 3 E 4 · COM A SENHA DO APP VAZADA

Três falsificações, três recusas

a) heartbeat **sem assinatura** dizendo que tok-12345678 está na sala	`x RECUSADA  SEM ASSINATURA — mensagem fora do envelope assinado`
b) assinado com a **chave do atacante**, mas dizendo ser tok-12345678	`x RECUSADA  IDENTIDADE TROCADA — assinou com a chave de tok-4481657a, mas diz ser tok-12345678`
c) assinado como ele mesmo, **de casa**, com token inventado	`x RECUSADA  TOKEN VENCIDO — o token 1a2b3c4d não é o que a anc-pc-1 anuncia agora`

Esquerda: **python ataques.py forjar** (janela 3). Direita: o que o **servidor** imprimiu (janela 2), conectado ao EMQX de verdade. Os mesmos casos são testes automáticos.

O CAMINHO DE VOLTA · O CELULAR TAMBÉM CONFERE

Ninguém fala pelo servidor nem pelo admin

tela da turma · cartão

Segurança

Canal: TLS (wss) com usuário **presenca-app**

Eu sou **tok-…** · chave d04ab232…

Recusei 2 mensagem(ns) falsa(s) — última: estado: chave desconhecida

Turma recusada: turma assinada por outro administrador

`ataques.py estado` publica “**PRESENTE, 100 %**” no tópico do aluno: sem assinatura e com a chave do atacante. A tela não muda; o cartão conta 2 recusas.

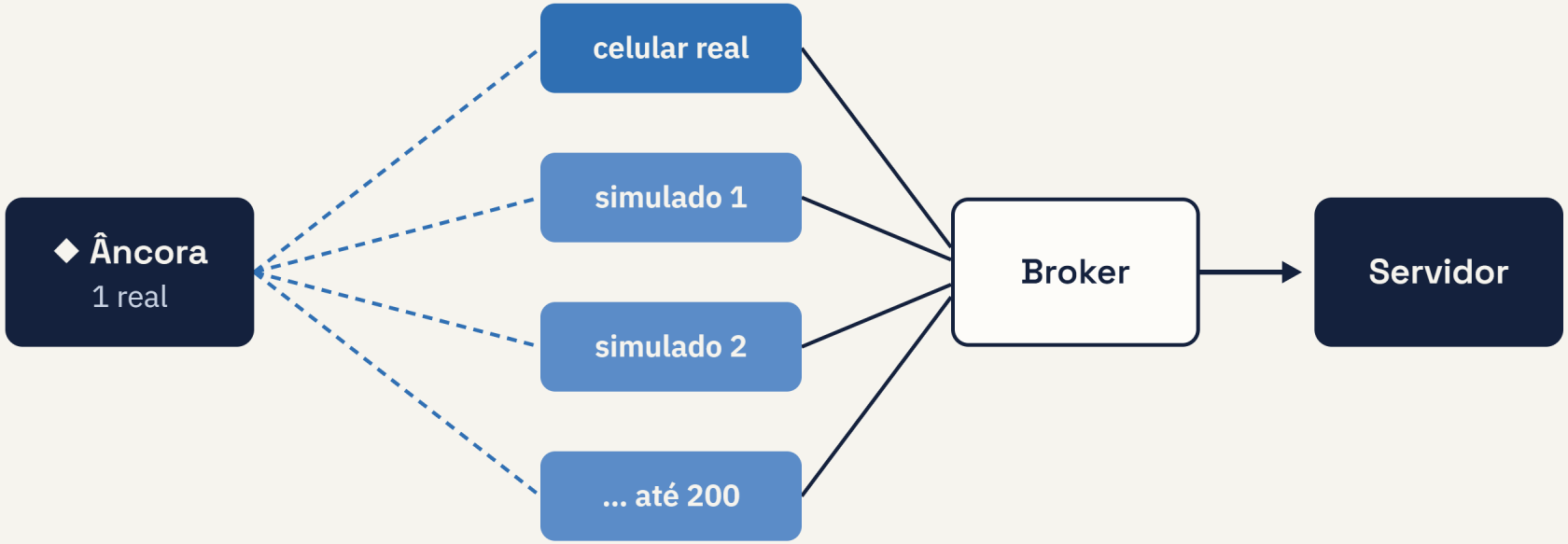
`ataques.py turma` publica a turma com o limiar de 1 m em **−100 dBm** — todo mundo pareceria perto, até de casa. Recusada: não é a chave do admin.

Como o celular sabe?

A chave pública do servidor vai fixada no APK. A do admin é guardada no pareamento (código ou QR, uma vez).

FAZER FUNCIONAR A REDE DE SENSORES

Cada celular é um sensor; o servidor junta todos



Funcionando

1 âncora, o celular real e celulares simulados, cada um com a sua chave.

Pronto para mais

o heartbeat leva até 3 âncoras, cada uma com o seu token e segredo.

Falta

3 âncoras físicas e a malha ESP-NOW entre elas.

ALÉM DO PEDIDO · QUANTOS DISPOSITIVOS AS ÂNCORAS MONITORAM?

Nenhum e todos: a âncora é um farol

Ela só transmite. Um anúncio Bluetooth é ouvido por **todos os celulares em volta ao mesmo tempo** — como uma lâmpada é vista pela sala inteira. Quem mede é cada celular, com o próprio rádio.

sem limite

de celulares por âncora

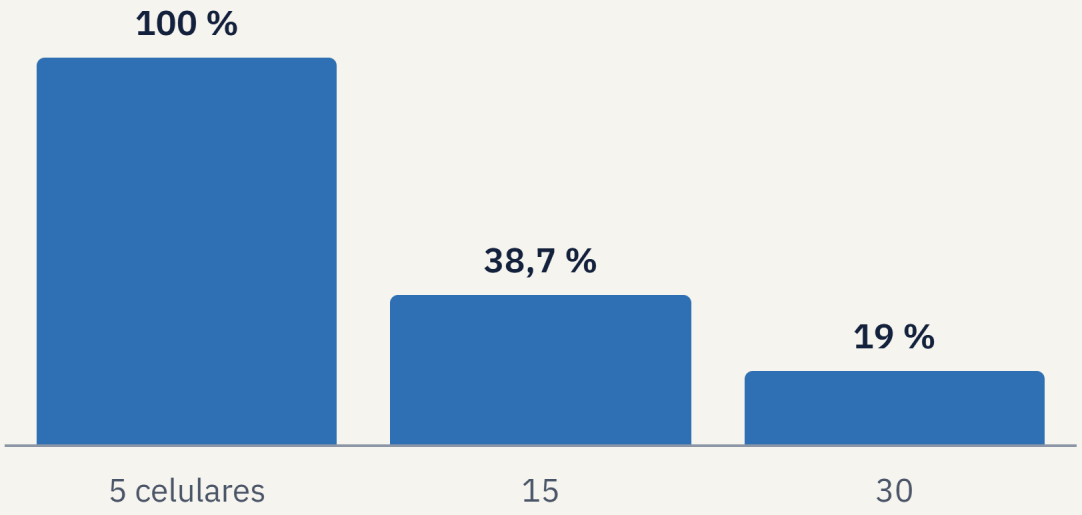
o limite está na volta

broker e servidor — e isso foi medido →

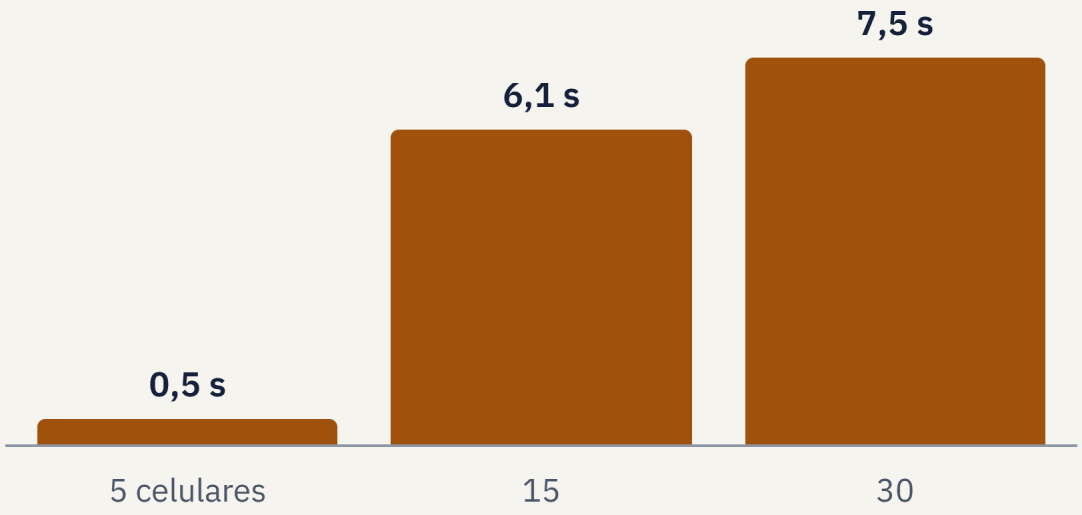
TESTE DE CARGA · BROKER EMQX REAL · HEARTBEAT A CADA 0,5 S

Até 5 celulares, tudo volta em meio segundo

Respostas que voltaram (%)



Ida e volta, mediana (segundos)



O servidor sozinho, sem rede: **0,4 ms por mensagem ≈ 2.400 mensagens/s**. Então a fila acima de 5 celulares **não vem do servidor**.

Presença · Marco 3 · 23 · python carga.py 5 / 15 / 30 (cada celular com a sua chave, assinando) · python carga.py 200 --sem-rede

ONDE ESTÁ O LIMITE · E “SE NÃO, COMO VAMOS FAZER”

O broker gratuito: ~11 mensagens/s por conexão

```
sonda: um cliente publica 60/s, outro conta — sem servidor no meio
enviadas 600 (60/s) · recebidas 157 · nos 10 s: 10.7/s
```

~11/s

por conexão no EMQX Serverless. O servidor usa **uma** conexão.

60 s

é o heartbeat da aula real (0,5 s é só na demonstração).

≈ 650

celulares por conexão do servidor: 11 × 60.

Para mais: várias cópias do servidor dividindo os heartbeats

Assinatura compartilhada do MQTT: $share/servidores/.../heartbeat. O broker entrega cada mensagem a uma só cópia, e cada cópia tem a sua conexão. Ou o plano dedicado, sem esse limite.

ALÉM DO PEDIDO · DÁ PARA FAZER COM ARDUINO?

Sim: a mesma âncora num ESP32

```
// esp32-ancora.ino
void loop() {
  janela = time(nullptr) / 12;
  if (janela ≠ janelaAtual) {
    janelaAtual = janela;
    anunciar(janela);  // PRS1|01|token
  }
}
// token: mbedtls HMAC-SHA256,
// o mesmo cálculo do servidor
```

1 · Mesmo anúncio

PRS1, número e token: o app e o servidor não mudam nada.

2 · Hora pelo Wi-Fi

NTP, para a janela de 12 s bater com a do servidor.

3 · Modo escuta

MODO_ESCUTA = true: conta os aparelhos em volta e mostra o RSSI.

Não gravado em placa — eu não tinha ESP32. O notebook faz o papel dele, com o mesmo anúncio e o mesmo token.

ALÉM DO PEDIDO · VALIDAÇÕES: DISTÂNCIA E QUANTOS POR RÁDIO

Medir: o processo inverso, só como ferramenta

Quantos aparelhos um rádio ouve

```
$ python escutar.py --alvo "Galaxy"
aparelhos ouvidos nos últimos 3 s: __
endereço · nome · RSSI · ~distância
```

O notebook (ou o ESP32 em modo escuta) varre os anúncios em volta. O celular precisa anunciar: app gratuito **nRF Connect** → Advertiser.

No sistema não se usa: a âncora passaria a saber quem está perto.

Do sinal para metros

```
d = 10 ^ ((RSSI(1 m) - RSSI) / (10 · n))
```

RSSI a 1 m: medido na calibração do admin. **n = 2,2:** estimado. Perto/longe não depende de n.

distância	RSSI medido
0,5 m · 1 m · 2 m	[__ · __ · __ dBm]
3 m · 5 m	[__ · __ dBm]

TESTES AUTOMÁTICOS · SAÍDA REAL

49 no servidor, 22 no app — todos passando

```
$ python -m pytest testes -v -k "segurança"
mensagem_sem_assinatura_e_recusada PASSED
carga_alterada_depois_de_assinada_e_recusada PASSED
assinatura_de_outro_topico_e_recusada PASSED
falar_em_nome_de_outro_aluno_e_recusado PASSED
token_de_outra_janela_e_recusado PASSED
token_inventado_e_recusado PASSED
heartbeat_da_fila_aceita_token_de_ate_48_s PASSED
respostas_saem_assinadas_pelo_servidor PASSED
```

49

servidor (pytest): permanência, contratos e segurança

22

app (npm test): inclui JavaScript e Python dando o mesmo pseudônimo

O QUE A SEGURANÇA NÃO RESOLVE

Limites declarados

Chave fora do chip

No armazenamento seguro do Android, cifrado pelo Keystore — mas um celular com root a extrai.

Senha do app no APK

Quem a extrai publica lixo (recusado) e lê pseudônimos. Próximo passo: ACL por tópico no EMQX.

Relay do token

Alguém na sala retransmitindo o token ao vivo para quem está em casa. Já era residual no projeto.

Rede e relógio

O broker só atende 8883 e 8084 (rede que bloqueia: hotspot). Âncora e servidor precisam de hora certa.

PERGUNTAS PROVÁVEIS · 1 DE 2 · SEGURANÇA

Se a senha do app vaza, acabou?

Não. Com ela se conecta, mas sem a chave privada de um aluno não se assina por ele, e sem ouvir a âncora não há token de agora.

E se gravarem uma mensagem e repetirem?

Na hora: mesmo eventId, vira REPETIDA e não soma. Depois: o token venceu. Mudar o número quebra a assinatura.

Por que não só TLS?

TLS esconde o conteúdo no caminho, mas não diz quem fala. O broker público também tinha wss e era aberto.

Por que Ed25519 e não HMAC para o aluno?

HMAC exige o mesmo segredo dos dois lados: o servidor poderia forjar o aluno. Com chave pública, só o aluno assina.

Por que não ficou no HiveMQ?

O público não tem senha; o Cloud deixou de ter plano gratuito (cartão, ~US$ 249/mês). O EMQX dá TLS, senha e ACL de graça.

E se o professor trocar o celular de aluno?

Celular novo = chave nova = pseudônimo novo. Novo pareamento pelo QR, como o projeto já previa.

E se a rede da faculdade bloquear?

O broker só atende 8883 e 8084. Se a rede bloquear, o notebook usa o hotspot do celular; o celular já usa dados.

Por que a âncora não escuta o celular?

O celular teria de anunciar o tempo todo (bateria, rastreio) e a âncora saberia quem está na sala. Só usamos para medir.

RESUMO · A ENTREGA DO MARCO 3

O que mostramos

Segurança
quatro camadas, cada ataque testado e recusado.

Autenticação MQTT
um usuário e senha por papel no EMQX.

Atuação + processo inverso
os dois lados produzem e consomem; o celular atua.

Canal criptografado
TLS obrigatório: só as portas 8883 e 8084.

Todo dispositivo diz quem é
celular, âncora, servidor e admin, cada um com a sua prova.

Rede de sensores
âncora + celulares + servidor, ponta a ponta.

Próximos passos: medir a distância de verdade, gravar o ESP32, três âncoras físicas, ACL por tópico.